Knowledge is Power: A Knowledge Graph-Based Approach for Mobile Malware Traceability Analysis

Yao Zhang, Guangquan Xu, *Member, IEEE*, Ruitao Feng, Xiaohong Li, Sen Chen, Zhenchang Xing, Yude Bai, Yongqiang Lyu, Wei Gong, *Senior Member, IEEE*, Xibin Zhao, *Member, IEEE*.

***Abstract*—The prevalence of Android malware has brought forth the issue of traceability in malware analysis, prompting the need for exploration. Establishing connections between newly discovered malware and existing data can shed light on the traceability analysis and underlying reasons behind the malware. However, real-world analysis of malware traceability is intricate and time-consuming due to the vast volume of data, often requiring manual examination and lacking explanatory results. Hence, a comprehensive automated malware tracing framework is urgently needed to provide detailed insights into traceability identification and explanatory capabilities.**

This paper introduces a knowledge graph-based approach utilizing partial API call graphs with semantic and behavioral features to uncover traceability relations among malware and generate explainable results. The approach is based on a dataset comprising over 20,000 labeled malware samples from a decade, addressing complexity through prior knowledge utilization and a branch pruning method for call graphs. This reduces computational complexity and enhances precision in determining traceability relations. Rigorous evaluation and validation were conducted, assessing the system's effectiveness in tracking malware through extensive experiments and results confirmation with further analysis. The system's ability is validated by effectiveness, soundness, and practicality to demonstrate the value of approach design and its real-world applicability for security professionals.

***Index Terms*—Mobile malware, traceability, knowledge graph, journal, API call graph, explainable analysis.**

I. Introduction

In recent years, Android has become the dominant mobile operating system. However, malicious actors have leveraged its open and flexible ecosystem to create and distribute increasingly sophisticated mobile malware and variants. This surge has driven the need for efficient methods to analyze the mechanisms underlying malicious behavior. Accordingly, numerous malware-analysis researches have been carried out, including static [1] and dynamic [2] program analysis, as well as machine- [3], [4] and deep-learning–based classifiers [5]–[7]. These methods pursue different objectives—defining malware families [8], identifying malicious behaviors [9], [10], and mining specific implementations [11], [12].

Traceability Analysis: Despite their effectiveness, existing malware detection research still lacks explicit modeling of malware evolution and standardized, semantically evidence units for explanation. To further understand the lineage and provenance of Android malware, researchers have increasingly turned to traceability analysis. In particular, several studies trace malware origins and propagation by identifying implementation patterns across various samples [13], [14]. While effective at improving analysis efficiency, many of them still rely heavily on expert knowledge, resulting in substantial manual effort.

Knowledge Graph Based Malware Analysis: To systematically characterize the behavioral semantics and evolutionary patterns of malware, knowledge graphs have been adopted to analyze malware behaviors. Existing graph-based studies typically unify diverse unstructured features into a common semantic and data structure [15], [16] to uncover latent associations and behavioral patterns [17], [18]. For example, ArgusDroid structures the semantic relationships between permissions and APIs in a knowledge graph to help identify variants of the same family [19]. Likewise, MalKG leverages a graph to link unstructured CTI and enable queryable analysis and graph-based inference [20]. However, these approaches generally overlook temporal (timeline) relationships among malware, and the large-scale nature of real-world program data makes modeling malware evolution extremely challenging.

Genome Based Approaches: In order to modularly characterize reuse, mutation, and recombination and to support knowledge-graph integration, prior research has proposed gene-based methods for studying malicious software [21], [22]. These approaches analyze the sensitive behavioral features of applications and capitalize on gene characteristics to represent fine-grained, reusable components of malware. Specifically, some studies extract data-flow dependencies as genetic evidence [23], and minimal statement subsequences [24], as well as API invocation patterns [25] as gene information. However, due to the limited semantic richness of these representations, these works still remain

(Yao Zhang and Guangquan Xu contributed equally to this work.) (Corresponding authors: Ruitao Feng, Xiaohong Li.)

Yao Zhang, Guangquan Xu, Xiaohong Li, and Yongqiang Lyu are with School of Cybersecurity of Tianjin University, Tianjin 300350, China (e-mail: zzyy@tju.edu.cn, losin@tju.edu.cn, xiaohongli@tju.edu.cn, lyuyq@tju.edu.cn).

Ruitao Feng is with the Faculty of Science and Engineering, Southern Cross University, Gold Coast, Bilinga QLD 4225, Australia (e-mail: ruitao.feng@scu.edu.au).

Sen Chen is with the Nankai University, Tianjin 300350, China (e-mail:senchen@nankai.edu.cn).

Zhenchang Xing is with the Research School of Computer Science, Australian National University, Canberra ACT 2600, Australia (e-mail:zhenchang.xing@anu.edu.au).

Yude Bai is with School of Software, Tiangong University, Tianjin 300387, China (e-mail:baiyude@tiangong.edu.cn).

We Gong is with the School of Computer Science and Technology, University of Science and Technology of China, Hefei, Anhui 230052, China (e-mail: weigong@ustc.edu.cn)

Xibin Zhao is with Beijing National Research Center for Information Science and Technology (BNRist), Key Laboratory for Information System Security, Ministry of Education (KLISS), School of Software, Tsinghua University, Beijing 100084, China (e-mail: zxb@tsinghua.edu.cn)

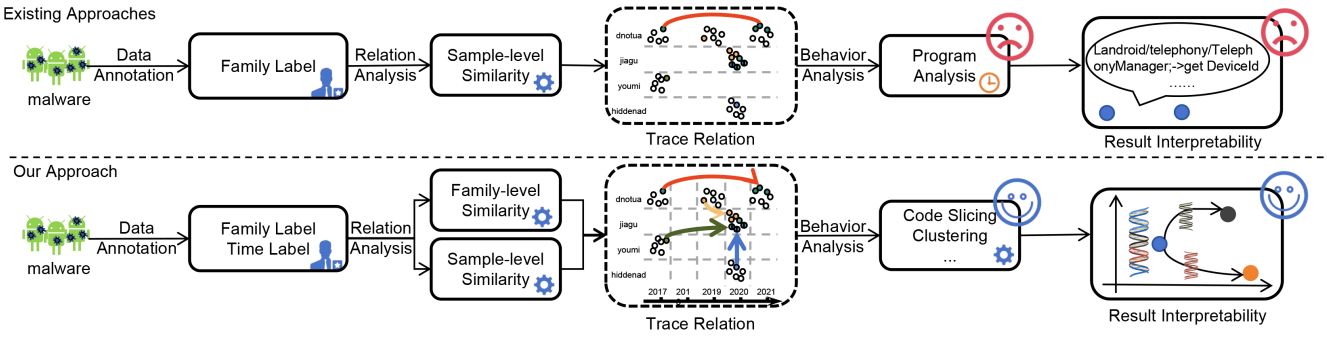


Fig. 1. The Novelty Demonstration of Automated Malware Traceability Analysis

at the level of function clones or isolated API fragments, lacking data/control-flow context and behavioral semantics, which makes them difficult to serve as evidence for explaining each lineage edge in malware traceability.

Challenges: In summary, existing work still faces several challenges: (1) High cost and unclear outcomes at scale. In real-world settings, the massive volume of malware samples makes traceability time-consuming, resource-intensive, and hard to present as clear evolutionary chains. (2) Weak interpretability due to featureless traceable units. Lacking semantically rich, behavior-aware evidence units prevents qualitative and quantitative characterization of evolution, undermining the credibility of results. (3) Bias from overreliance on family labels. Methods that depend mainly on family labels overlook other malicious behaviors and, in complex code-reuse scenarios, can misattribute variants to a single source.

Design and Solutions: To overcome the above challenges, we introduces a malware traceability analysis framework that generates knowledge graphs to study malware traceability and composition. Specifically, 1) to address the high cost and lack of clarity in large-scale traceability, we construct a structured malware representation that combines semantic features, and behavioral features extracted from the call graph (CG) and static call graph (SCG), thereby reducing analysis complexity and providing a clear data foundation for subsequent visualization and automated tracing. 2) To remedy the lack of semantically rich, interpretable traceable units, we introduce Sensitive Subgraphs (SSGs) and treat them as malware "genes," capturing behavior-level evidence that enables fine-grained explanations of evolutionary relations and auditable traceability. 3) To alleviate overreliance on family labels and the difficulty of discovering cross-family evolution, we build knowledge graphs at both the family and sample levels and couple them with machine-learning–based clustering and temporal association analysis, thereby reducing label dependence and unveiling complex cross-family lineage chains.

Taking Figure 1 as an example, we augment data processing with timestamps (packaging time) and family labels to introduce a temporal dimension, and perform two-level traceability (family and sample). This enables construction of a cross-family knowledge graph that depicts evolutionary paths over time. Instead of traditional program analysis, we treat malware behaviors as genetic material, providing precise qualitative and quantitative explanations of lineage and evolution.

Evaluations and Contributions: We evaluated the effectiveness of our approach using 20,142 Android malware samples collected from AndroZoo between 2011 and 2021. To ensure data quality and fairness, all samples were verified using VirusTotal and labeled via AVclass, where each sample was confirmed by at least ten antivirus engines. We compared our results with baseline family relationships, observed the identified evolutionary chains across 13 representative malware families, and examined fine-grained behavioral inheritance among them. The results demonstrate that our approach can efficiently capture cross-family evolution and provide auditable, interpretable evidence for each lineage relation, confirming the effectiveness and practicality of the proposed framework. To summarize, we make the following contributions:

1) A framework for traceability analysis that involves establishing a knowledge graph utilizing prior knowledge of malware and the underlying implementation of malicious behavior.
2) Leveraging the knowledge graph built upon a vast malware dataset, our all-in-one approach enables the discovery of highly comprehensive traceability relations while providing precise and in-depth explanations of the root causes.
3) Motivated by the evolutionary dynamics of software in real-world programs, we introduce the concept of family–malware–gene to standardize the methodology for malware traceability analysis.
4) We demonstrate the capability of our approach in traceability analysis and explanation of the captured relations on large-scale real-world malware data.

II. Approach

A. Overview

The framework (Figure. 2[1]) comprises three modules: malware representation, coarse-grained traceability analysis, and fine-grained traceability explanation. The first module uses two types of features to characterize malware, semantic information, and malicious behavior. The second module conducts traceability analysis between families and samples. The third

[1]SCG and SSG stand for Sensitive Call Graph and Sensitive SubGraph, respectively.

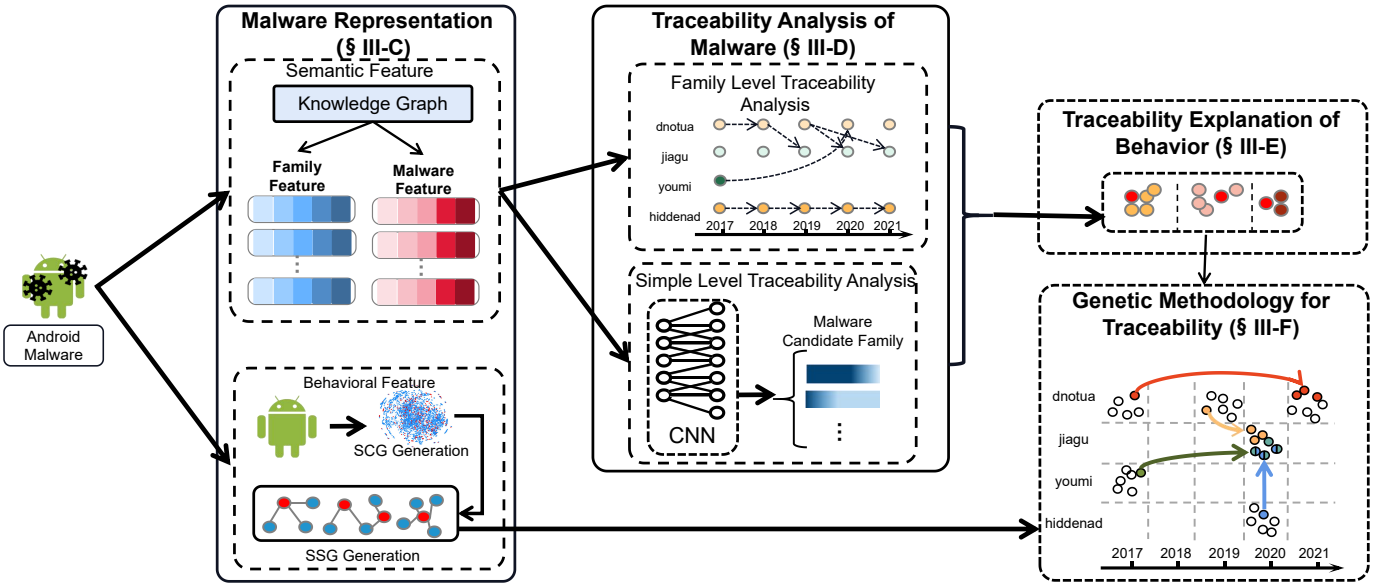


Fig. 2. The Framework for Malware Traceability Analysis

module explains traceability at the behavior level along with malware genetic methodology.

B. Baseline Approaches

1) Pre-experiment of Data: To gather a large-scale malware dataset suitable for evaluating our approach's capability, data preferably with timestamps and a long time span is essential. We list the key selection criteria for collecting the malware data as follows:

- **Time Distribution:** The samples must be evenly distributed over the past ten years, from 2011 to 2021. This ensures a comprehensive analysis of traceability over a significant time period.
- **Family Labeling:** To facilitate family classification, the samples must have been verified by VirusTotal [26]. This allows us to add family labels to the malware samples.
- **High Confidence Malware:** To ensure the inclusion of reliable malicious samples, we employ AVclass [27] for filtering. To be considered for our dataset, a sample must be detected as malicious by at least 10 virus engines within VirusTotal.

To meet practical demands, we target 2,000 samples per year as a baseline for data collection. To ensure dataset balance, we compared the family labels of samples from each year to ensure their family numbers and distributions were similar. For instance, in 2012, there were 101 sample families, and we selected 88 sample families with similar distributions of mainstream families like "kugu", "adwo" and "gappusin". We collected 20,142 malware from Androzoo [28].

2) Key Factors in Traceability Analysis: During the data collection and literature study, several intriguing insights about potential affecting factors to the results of traceability analysis were derived.

- **Limited Scope of Traceability:** Considering connections between malware from different families is vital for accurate traceability analysis. Shared characteristics [29], [30] and code reuse [31] can lead to misinterpretations.
- **Improper Traceable Unit:** Defining the traceable unit correctly is essential. A broad definition may lead to false relations, while a narrow one can miss crucial connections.
- **Unclear Methodology:** Ambiguous or poorly defined methodologies can introduce bias and errors, reducing the credibility of traceability analysis.

3) Initial Design of Knowledge Graph: Leveraging a vast dataset spanning a significant timeframe, we employ the knowledge graph as the core framework to present our malware traceability analysis outcomes. We create a graphical data structure that effectively characterizes the malware by organizing the data, incorporating relevant features, and utilizing predefined labels. This approach facilitates exploring and visualizing relationships between diverse and multifaceted malware samples within predefined dimensions.

C. Malware Representation

This module has two units: semantic feature providing quick context for traceability relations and behavioral feature for precise characteristics.

1) Semantic Feature Characterization: The semantic feature characterization unit represents malware's basic object units, which consist of sensitive permissions and APIs.

① **Feature Vectorization:** To create a family-level graph, we utilized 60 commonly-used sensitive permissions and 95 API features [32]–[34] to represent malware and families. The vectorized representation of features is to abstract the relation of triples in the malicious family knowledge graph as text semantics [35]. We divide it into two entities: malware family and malware samples.

- For each family, the features of each sample are mapped to the joint vector space. Each dimension in the vector represents the occurrences of a feature in the family. If a sample

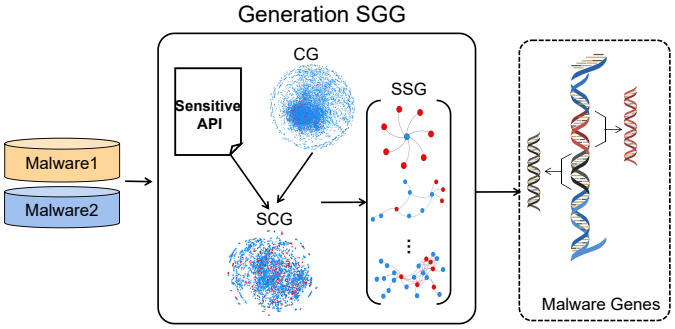


Fig. 3. Workflow of SSG Generation

in a family contains a feature, then the value corresponding to the feature in the vector matrix is incremented by 1.

- For each sample, it is the same operation, except the characterized subject is changed from malware family to sample. When a feature appears in a sample, the position of the feature corresponding to the vector is set to 1.

② **Weight Assignment for Features:** To distinguish the importance of features, we use the theorem of TF-IDF [36] to calculate and assign weights ($\omega(s, f)$: the weight of sensitive feature s in family f) to features.

$$\begin{cases} \omega(s,f) = per(s,f) * log\frac{allNum}{Totalnum(s)} * log\frac{allTypeNum(f)}{TotalTypenum(f,s)}, \\ per(s,f) = \frac{num(s,f)}{fnum(s)}. \end{cases} \tag{1}$$

where s is a feature to build the knowledge graph. $num(s, f)$ is the occurrences of feature s in family f. $per(s, f)$ is the percent of samples that invoke the feature s in family f. $fnum(s)$ denotes the number of samples in f. $allNum$ denotes the number of all samples. $Totalnum(s)$ denotes the number of samples containing feature s. $allTypeNum(f)$ denotes the number of all features contained in the family f. $TotalTypeNum(f, s)$ denotes the number of malware in the family f that contain feature s.

③ **Two ways to use:** We can get the feature representation of the malware sample and family through the above two steps by combining vector and weight. For the malware family, we take the malware family as the main body and the respective features under it as the features.

2) Behavioral Feature Characterization: Characterizing features that comprehensively reflect the implementation of malicious behaviors at a more fine-grained granularity is essential in representing behaviors. We utilize sensitive APIs as fundamental elements. However, relying on sensitive APIs may lead to massive false alarms and require additional analysis efforts due to the various contexts in which these APIs are used. We adopt call graphs as the malware representation instead of discrete or sequential features to address this. We prune the graphs using a depth value to manage the computational complexity of complete call graphs. This allows us to generate partial graphs centered on sensitive APIs, providing a more comprehensive outline of the malicious behaviors. The detailed workflow is illustrated in Figure. 3[2]. Specifically, for each malware sample, we first build a call graph (CG) from its bytecode and project it to a Sensitive Call Graph (SCG) by keeping only sensitive APIs and their nearby callers. We then apply community detection on the SCG and retain communities that contain at least one sensitive API as Sensitive SubGraphs (SSGs). In our design, each SSG is treated as a malware gene, and the "genome" of an APK is simply the set of its SSGs/genes used in our subsequent traceability and evolution analysis.

① **SCG Generation:** The entire Android application's function call relation is represented in the CG, offering detailed features of malware. However, due to its massive size (approximately 2 TB for the dataset), direct use of the CG incurs significant time and computational costs. To address this, we focus on the subset of sensitive APIs, which are crucial for most malware to access sensitive information and perform malicious tasks. By highlighting these sensitive APIs related to malicious behaviors in the CG, we create the SCG ($SCG = (V', E')$), which is a maximum subgraph of the CG ($CG = (V, E)$), by utilizing a set of 9,730 sensitive APIs [37]. In the process, we prune the non-parent nodes of sensitive API nodes in the node-set V of the CG, effectively reducing the complexity of graph analysis, as described by Equation 2.

$$\begin{cases} V_g = \{V_j | \exists V_i \in V_s, 0 < dist(V_j, V_i) < n, V_j \in V\}, \\ V' = V_s \cup V_g, E' = (V' \cup V') \cap E. \end{cases} \tag{2}$$

where V_s represents the sensitive API node collection, and function $dist(V_j, V_i)$ returns the shortest path from node V_i to node V_j.

② **SSG Generation:** After data processing, each sample is represented as an SCG. While some SCGs have distinct structures, there are also similarities among SCGs from different samples. Directly matching two SCGs is inefficient due to the problem of subgraph isomorphism, which is known to be NP-complete [38].

To accurately identify similar behaviors between different samples and reduce computational complexity, we divide the SCG into smaller subgraphs. Previous research has shown that the CG of malware exhibits a community structure in complex networks, where vertices form communities with high internal connections and sparse connections between communities. We apply the infomap [39] community partition algorithm to the SCG and observe that the modularity Q value falls between 0.6 and 0.8, confirming the existence of a community structure in the SCG.

Not all subgraphs generated by community division are relevant to sensitive data and extracting malicious behaviors. We retain subgraphs that contain at least one sensitive API node and define them as SSGs. Each SSG is assigned a weight $w(SSG)$, indicating its importance relative to the family. To calculate $w(SSG)$, we use the method proposed in § II-D1 that assigns different weights to various sensitive APIs, as described by Equation 3.

$$w(SSG) = \sum_{V_i \in V_s(SSG)} w(v_i, f). \tag{3}$$

[2]CG, SCG, and SSG stand for Call Graph, Sensitive Call Graph, and Sensitive SubGraph, respectively.

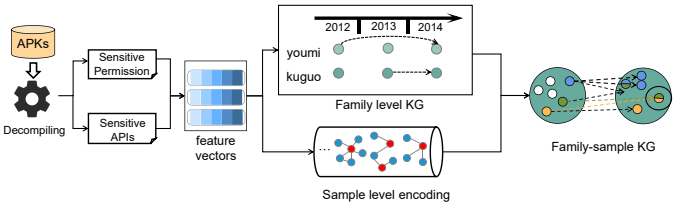


Fig. 4. Knowledge graph Construction

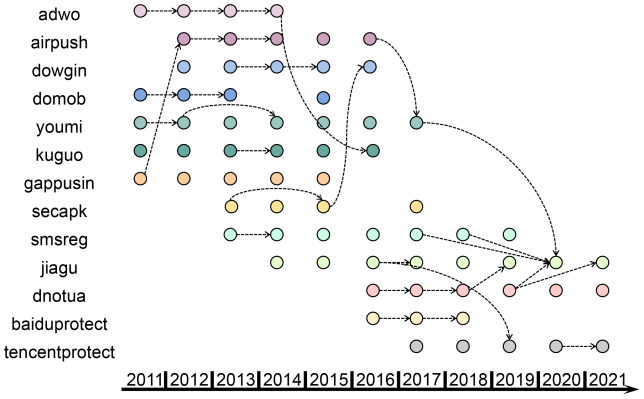


Fig. 5. Family-level Knowledge Graph

where $V_s(SSG)$ represents a collection of sensitive API nodes in the SSG and $w(v_i, f)$ represents the weight of node v_i in its family f.

D. Traceability Analysis of Malware

Next, according to the semantic features extracted from malware, we design a knowledge graph-based approach to conduct a coarse-grained traceability analysis of malware. As shown in Figure 4, our approach constructs a Family-Sample Knowledge Graph by clustering APKs into families based on feature similarity and linking these families with temporal and evolutionary relationships. This graph enables us to trace the origins and connections between different malware families and their samples, providing valuable insights into malware propagation and evolution.

TABLE I
EVALUATION METRICS FOR DIFFERENT THRESHOLDS ON COSINE DISTANCE

Cosine Distance	Precision	Recall	F1-Score	Accuracy
0.65	0.73	0.80	0.77	0.78
0.7	**0.80**	**0.85**	**0.82**	**0.83**
0.75	0.84	0.78	0.81	0.82
0.8	0.9	0.74	0.79	0.8

1) Family-level: Through two steps in §II-C1, we can get the vectors of which each representing a family and the weights of all features. We then calculate the cosine distance between any two families. To select an appropriate threshold that can accurately capture families with evolutionary relations, we compare the performance under different cosine distance thresholds based on our manually constructed dataset (See Section III-B). As shown in Table I, a threshold

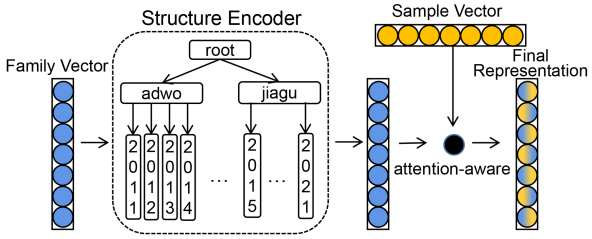


Fig. 6. Hierarchical Feature Structure

of 0.7 provides a favorable operating point: it achieves the highest Recall (0.85), F1-score (0.82), and Accuracy (0.83), while maintaining a competitive Precision of 0.80, and is therefore adopted as our threshold. By observing the result on our dataset, 0.7 (selected threshold) is the gold number of the threshold to accurately capture the families with evolutionary relations. When the distance between the two families exceeds 0.7, There is a genetic relation between the two paired families, and this relation is reflected in our family graph as a path. Finally, we establish a derivation knowledge graph at the coarse-grained family level, which can show the trend of evolution across the timeline. As shown in Figure. 2, the abscissa is the time axis, and we draw the existence of each family on the timeline. Figure. 5 demonstrates the traceability relations among several representative families in the obtained knowledge graph.

2) Sample-level: In order to empower the capability of the knowledge graph to investigate the precise relation between samples, We refine the granularity to the sample level. Existing malware family classifications ignore the impact of time changes on the implementation of malware belonging to the same family [37], but in fact, these impacts are widespread and essential to evolution analysis, such as changes in API definition and calling mechanisms [40]. Hence, we need a novel approach that can classify in two dimensions, i.e., the family feature and the released year simultaneously. To this end, we establish a hierarchical structure to fuse the family and year tags and set up an encoder to perceive the relation between tags. Adopting a multi-label attention mechanism, the final input vector is generated from the encoded family and sample feature vectors and fed into a CNN to train the classifier. The partial input generation overview is shown in Figure. 6. Note that, although graph neural networks are a common choice for generic graph-structured data, our encoder operates on a fixed-size, grid-structured representation (a multi-channel feature matrix over family, temporal, and behavioral signals), for which a CNN with attention is a natural and efficient fit. Running GNNs directly on the full, large-scale knowledge graph would require expensive message passing on a sparse and noisy graph and is thus less scalable in our setting. The final input generation overview is shown in Figure. 6

① **Hierarchical Family Information Encoding:** First, we represent the global family information as a graph. Family and year information set is defined as the nodes $V = \{v_1, \ldots, v_D\}$, where D represents the number of all types of family and year information in the set. The edge between nodes represents the relation between the family and year. The set of edges

is defined as $E = \{(v_a, v_b)|v_a \in V, v_b \in child(v_a)\}$, which is the top-down collection of the family type and year information. Hence, the hierarchical structure of family types with year numbers is defined as a directed diagram $G = (V, E)$, whose rough structure is shown in the dashed box "Structure Encoder" in Figure. 6.

We assume that there is a path between the parent level node v_a and the child level node v_b in the hierarchical structure, and the edge between the two nodes has a weight $a(v_a, v_b)$, which expresses the degree of association between the nodes, whose value is the conditional probability between two sample types:

$$a(v_a, v_b) = \begin{cases} P(v_b|v_a) = \frac{P(v_b) \cap P(v_a)}{P(v_a)} = \frac{P(v_b)}{P(v_a)} = \frac{N_b}{N_a}, \\ 0. \end{cases} \tag{4}$$

where $P(v_a) \cap P(v_b)$ is the probability that v_a, v_b occur simultaneously. P is the probability of the label appears in the dataset. N is the number of times the label appears in the dataset. The sum of the conditional probabilities of all sub-labels of the v_a node is 1. When there is an edge between v_a and v_b, $a(v_a, v_b) = \frac{N_b}{N_a}$, otherwise, $a(v_a, v_b) = 0$.

The obtained weighted directed graph perfectly expresses the prior knowledge we need for constructing the sample-level knowledge graph. To encode the graph, we first calculate $u(v_a, v_b)$, which is the feature representation of the parent node of v_a (i.e., v_b). The hidden state output of node v_a represents the physical type representation of the $h(v_a)$ corresponding to its hierarchical structure.

$$\begin{cases} h(v_a) = RELU(v_a^T \odot u(v_a, v_b)), \\ u(v_a, v_b) = a(v_a, v_b) v_b. \end{cases} \tag{5}$$

② **Hierarchical Family-sample Perception:** Since we want to take both the prior knowledge (i.e., family and year information) and key implementation features in classification, an attention-aware deep neural network [41] is applied to perceive the final vector representation of the overall sample information. We take the encoded family-year vector obtained in § II-D2 and the sample vector obtained in § II-D1 as inputs and output a final vector. Specifically, by fusing the sample vector with the family-year vector encoded by the hierarchical structure encoder, the obtained feature Z is calculated in the following Equation 6:

$$Z = h(v_a S^T). \tag{6}$$

where S is the sample feature that has passed the attention network. $h(v_a)$ is the label representation.

We map the final features to low-dimensional vectors for final classification and input them into a CNN [42]. For every family, two-thirds of the malware is used as the training data, and the rest is used as the test set. With the pre-trained model, we can get the candidate families of a given sample and the probabilities that represent the sample belonging to the families.

E. Traceability Explanation of Behaviors

After identifying the traceability direction, the objective is to explore the root cause of the high similarity among malware samples, which could be due to code reuse and other factors. To achieve this, we perform traceability analysis at a finer-grained level by clustering the malicious behaviors from different samples, based on the features extracted in § II-C1. Additionally, we define key implementation characteristics of the behavior as "malware genes" to standardize the explainability of the results. The detailed workflow for this process is depicted in Figure. 7.

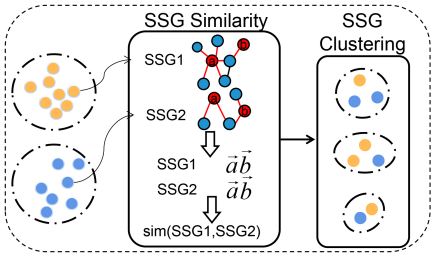


Fig. 7. Workflow of SSG Clustering

1) SSG Similarity: Because malicious behavior is mainly reflected by sensitive APIs, we consider sensitive API nodes of SSG as the key factor when calculating SSGs similarity. The algorithm mainly depends on the structure equivalent of SSGs and calculate the similarity of SSGs through three steps.
① **Building distance matrix:** First, construct the corresponding distance matrix for each SSG, mainly used to describe the relation between sensitive API nodes in the graph. The distance $M_{SSG}(i, j)$ of the SSG is obtained by:

$$M_{SSG}(i, j) = \begin{cases} dist(v_i, v_j), & v_i, v_j \in V_S(SSG), \\ \infty, & otherwise. \end{cases} \tag{7}$$

where $V_S(SSG)$ is a collection of sensitive APIs in SSG.
② **Sensitive API Nodes' Similarity Calculation:** After constructing its corresponding feature for each SSG, we can effectively transform the high-complexity subgraph matching computation problem into a feature similarity computation problem, which is much simpler. For the features of two SSGs, we stipulate that the more sensitive API nodes the two SSGs have in common, and the more similar the topological relations of these sensitive API nodes, the more similar the two SSGs.

First, According to the distance matrix, a t-dimensional vector $\overrightarrow{vec(v_i, SSG)}$ is constructed for each sensitive API node, t is the number of sensitive API nodes in the graph.

$$\overrightarrow{vec(v_i, SSG)} = \frac{1}{M_{SSG}(i, 1)}, \ldots, \frac{1}{M_{SSG}(i, t)}. \tag{8}$$

The similarity of the sensitive API nodes v_i in the subgraphs SSG1 and SSG2 is expressed as $ns(v_i)$, using the cosine similarity calculation equation to obtain.

$$ns(v_i) = cos(\overrightarrow{vec(v_i, SSG_1)}, \overrightarrow{vec(v_i, SSG_2)}). \tag{9}$$

③ **SSG Similarity Calculation:** Since different sensitive APIs are given different weights, the calculation of SSG similarity relies on the weighted average of the similarity of the same sensitive API nodes in two SSGs, which is calculated by Equation 10.

$$sim(SSG1, SSG2) = \frac{\sum_{v_i \in V_s(SSG1) \cap V_s(SSG2)} w(v_i, f) \cdot ns(v_i)}{\sum_{v_i \in V_s(SSG1) \cup V_s(SSG2)} w(v_i, f)}. \quad (10)$$

TABLE II
EFFECT OF DIFFERENT THRESHOLDS ON SSG SIMILARITY

Threshold (SSG Similarity)	Precision	Recall	F1-Score	Accuracy
0.70	0.82	0.90	0.86	0.84
0.75	**0.88**	**0.87**	**0.88**	**0.87**
0.80	0.91	0.80	0.85	0.83
0.85	0.94	0.68	0.79	0.80

2) SSGs Clustering: We define N clusters, where N represents the number of SSGs in the test sample. Each cluster consists of only one SSG from the test sample. We then compare the similarity between the SSGs of the test sample and those of the samples in the candidate families. Using a threshold, we cluster the SSGs that represent the same malicious behavior. To select a appropriate threshold, we added a sensitivity study over different SGG similarities. As shown in Table II, lowering the SSG similarity threshold (e.g., 0.70) yields higher recall but introduces more noisy matches, while higher thresholds (0.80/0.85) become overly strict and substantially reduce recall. The threshold of **0.75** achieves the best balance with the highest F1-score (0.88) and accuracy (0.87), so we adopt it as the default setting. If the similarity between SSGs exceeds the threshold[3], the candidate SSG is added to the cluster. If multiple clusters meet the condition for SSG addition, we select the cluster with the highest average similarity.

As a result, we obtain sub-graphs representing malicious behaviors that are similar to those of the test sample. These sub-graphs are obtained through the clustering results based on the SSG analysis. By analyzing the time axis, we can determine the clusters of sub-graphs related to the samples over time, providing insight into the evolutionary relationships between the malware samples.

By performing cluster analysis of SSGs, we can identify the clusters of sub-graphs related to the samples to be traced and discover similar sensitive sub-graphs from different malware samples. Based on this information, we can determine the malware samples that share a relation with the sample to be traced. This allows us to infer the source of the variant based on the context of their appearances.

F. Genetic Methodology for Traceability

Upon completing the malware families and samples analysis, we perform a meticulous traceability analysis at the behavior level using the method described in § II-E2. This analysis is not only an explanation of the coarse-grained traceability analysis results but also a confirmation of those results. However, to further explain the specific chain of traceability relations and its reasons, we need to investigate the specific software information that is the primary factor in this process. Malware genes, which act as the genetic material in malware, can fulfill our requirements effectively. These genes, similar to those found in organisms, are present within the malware itself and can express the malware's functions and information, and they are preserved during malware inheritance. The biological definition of a gene refers to the fundamental genetic unit that controls biological traits. In the case of malware, the traits are exhibited in the unique malicious behavior.

For malware, its gene can be understood as a partially abstracted CG of malware, which is the smallest functional unit that inherits malicious behavior and the primary genetic unit that controls the malicious function of malware. It is a relatively independent unit of genetic information that can be recombined in different modules in various ways and inherited by future generations. The malware gene in this paper is defined as the SSG possessed by malware. The malware genetic methodology helps to analyze the function of malware and also has a specific effect on the homology determination of malware. As a traceable unit medium in malware, malware genes can explain the evolutionary reasons by tracking the specific changes of malware genes, and reasonably explain the traceable results so that more effective measures can be taken in response.

[3]For our dataset, we set the threshold to 0.75.

III. EVALUATION

We assess our approach based on effectiveness, soundness, and practicality to demonstrate the real-world applicability of our malware traceability analysis framework.

A. Experimental Design

We aim to address three primary research questions (RQs).
RQ1 - Effectiveness: *How capable is our approach in conducting traceability analysis, and does it produce consistent traceability relations as expected?* We evaluate the effectiveness by identifying the traceability relations among large-scale data and validate if the results align with the expectation by design.
RQ2 - Soundness: *To what extent are the key designs in our framework valid, and how well does the genetic concept of malware empower the explainability of traceability relations?* We verify the validity of the designed techniques to capture traceability relations effectively and the feasibility of explaining with malware genetic methodology.
RQ3 - Practicality: *How practical is our approach in real-world scenarios, and how suitable is it for security experts and practitioners?* We evaluate the practicality of our framework by assessing its effectiveness in detecting and explaining traceability relations in a real-world setting to determine its applicability for security professionals and practitioners.

Note that, our approach relies on a curated list of sensitive APIs. Although Android APIs evolve over time, we mitigate

this threat by maintaining the list across versions and intersecting it with APIs observed in our 2011–2021 corpus. In practice, the evolution analysis is dominated by long-lived, frequently used sensitive APIs, and small changes in rare, version-specific calls have limited impact.

B. RQ1 - Effectiveness

To evaluate the effectiveness of our approach, we initially employ a family/sample-level knowledge graph to conduct a retrospective analysis of representative Android malware families, and then perform a behavioral-level traceability analysis based on the obtained results. Since there is no publicly available dataset with explicit ground-truth labels for malware family evolution, we follow the manual construction strategy adopted by Fan et al. in the FalDroid system and build an evaluation dataset specifically for evolution analysis on top of our AndroZoo-based corpus. Specifically, we randomly sample about 300 Android malware samples from AndroZoo that satisfy our VirusTotal and AVclass filtering criteria and submit each sample to VirusTotal to obtain multi-engine scan results. Two researchers with experience in malware analysis manually merge and normalize the family labels reported by different anti-virus engines, using string edit distance and family-name dictionaries from prior work as auxiliary references. For samples with large disagreements among engines, we further inspect their package names, certificate information, and existing analysis reports, and either assign them to a consistent family or discard them. After several rounds of manual filtering, we retain about 150 high-confidence samples covering 13 families. Based on the first-seen time recorded by VirusTotal, we manually verify and correct abnormal timestamps, aggregate samples into their corresponding calendar years, and keep only those family–year nodes that contain at least k samples for subsequent evolution modeling and evaluation. Although the resulting dataset is of moderate size, it is heavily curated and annotated by humans, which ensures relatively high reliability of both the family labels and the temporal information. The dataset is available on our website [43].

On top of this curated evolution dataset, we quantitatively evaluate our method against three representative baselines: a gene-based approach [24] (denoted as *Gene-based*), a graph-based evolution method *FalDroid* [37], a knowledge-graph-inspired variant identification method *ArgusDroid* [19], and a graph-based behavior analysis method *DroidMiner* [44] . Since the original Gene-based and DroidMiner papers do not provide publicly available source code, and *FalDroid* is not directly designed for malware evolution prediction, we re-implement and customize these methods for our setting. Concretely, we follow their feature extraction and graph/gene modeling procedures and adapt their representations to define similarity scores between family–year nodes under a unified evaluation protocol, so that all baselines can be fairly compared against our approach on the same evolution-link prediction task (RQ1). Given the absence of a real-world ground-truth dataset for Android malware traceability, our effectiveness evaluation thus focuses on whether the proposed framework can (i) achieve superior quantitative performance on evolution-link prediction compared with these baselines (RQ1), and (ii) uncover diverse and meaningful traceability relations across different families over a long time span, as analyzed in RQ2 and RQ3.

TABLE III
COMPARISON OF OUR APPROACH WITH REPRESENTATIVE BASELINES ON THE MALWARE EVOLUTION PREDICTION.

Method	Accuracy	Precision	Recall	F1	AUC
FalDroid	0.81	0.82	0.76	0.79	0.88
Gene-based	0.76	0.77	0.70	0.73	0.83
DroidMiner	0.74	0.75	0.68	0.71	0.81
ArgusDroid	0.79	0.82	0.78	0.81	0.89
Ours	**0.88**	**0.88**	**0.86**	**0.87**	**0.93**

To comprehensively evaluate the performance of our model, we adopt five complementary metrics: *Accuracy, Precision, Recall, F1-score,* and *AUC*. Accuracy measures the overall correctness of predictions, Precision quantifies the reliability of positive predictions, Recall captures the coverage of true positives, and F1-score balances Precision and Recall. In addition, AUC (area under the ROC curve) assesses the model's ranking ability across all possible decision thresholds, providing a threshold-independent evaluation.

Accuracy measures the proportion of correctly classified samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{11}$$

Precision evaluates the fraction of predicted positives that are truly positive:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{12}$$

Recall measures the fraction of actual positives that are correctly identified:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{13}$$

F1-score is the harmonic mean of Precision and Recall:

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \tag{14}$$

AUC summarizes the area under the ROC curve, where $\text{TPR} = \frac{TP}{TP+FN}$ and $\text{FPR} = \frac{FP}{FP+TN}$:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR}) \tag{15}$$

As shown in Table III, our approach achieves the best overall performance on malware evolution prediction, with the highest Accuracy (0.88), F1 (0.87), and AUC (0.93). Among the baselines, ArgusDroid and FalDroid performs the strongest, achieving F1 scores of 0.81 and 0.79 and AUC values of 0.89 and 0.88 respectively, while the gene-based method and DroidMiner lag further behind. This indicates that, although existing family-oriented, relation mining techniques, and gene-oriented techniques can be adapted to approximate evolutionary links, explicitly modeling family–year nodes on a knowledge graph with SSG-level semantics yields more accurate and robust evolution predictions.

Additionally, delving into the captured evolutions led us to unveil fascinating insights into the dynamic nature of Android malware.

- One malware family in a certain year may not inherit behaviors from the previous year. For example, the family "youmi" in 2014 is dissimilar to the one in 2013, with a cosine distance less than 0.60 (without an arrow). However, "youmi" directly evolves from 2012 to 2014 (with an arrow).
- API deprecation due to SDK update will cause the family to disappear, such as "secapk" cannot be found after 2017. Manual decompilation of "secapk" malware reveals that over 90% of them run on mobile devices with SDK versions lower than 6.0, possibly due to enhanced permission management after Android 6.0 was released [45]. In contrast, the most popular SDK versions in 2018 are 6.0 and 7.0 [46], covering around 60% of the market. This decline in usage of SDK versions below 6.0 correlates with the decreased prevalence of "secapk" malware, making it likely that the family ceased to exist after 2018.
- The behavior of a malware family can be derived from various families. For instance, the behavior of "jiagu" in 2020 appears similar to the behaviors of "youmi" 2017, "smsreg" 2017 and 2018, and "dnotua" 2019. The proportion of all malware families in "jiagu" increases significantly from 0.4% in 2013 to 53% in 2021. This suggests that some malware families, especially those that disappeared, might have merged into "jiagu", displaying a more diverse range of malicious behaviors.

Remark: *Our approach is effective in identifying traceability relations among 13 malware families within the scope of design through family/sample-level knowledge graphs and behavioral-level analysis. We observed families evolving, disappearing due to technological changes, and at times, incorporating behaviors from other families. These findings emphasize the importance of analyzing malware traceability as a complex and essential aspect of cybersecurity research.*

C. RQ2 - Soundness

We thoroughly examine the captured traceability relations to ensure consistent and accurate identification of relations at both family and sample levels, aligning with behavior-level analysis results. To showcase explainability, we investigate selected cases "genome" composition. This demonstrates the feasibility of explaining the root causes of captured traceability relations using the proposed malware genetic analysis methodology.

1) An In-depth Dissection of Traceability Relation: We analyze two representative cases of traceability analysis from the results of families "adwo" and "jiagu". We highlight the assistance of SSG-based fine-grained behavior analysis to confirm the observed trends and explore root causes.

① **Traceability Relation of "adwo".** From the family aspect, "adwo" is one of the most popular malware families which possesses 41% malware of the year 2011 in our dataset. However, this family cannot be found after the year 2014.

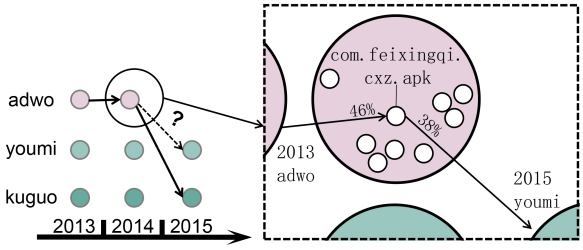


Fig. 8. Traceability Analysis from Family-level to Sample-level

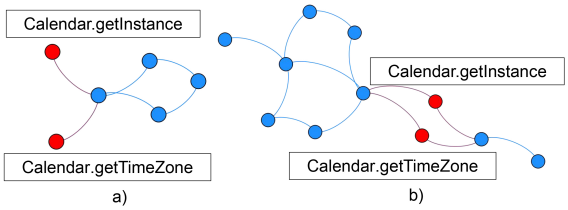


Fig. 9. SSGs in the Same Cluster

We select the malware "com.feixingqi.cxz.apk" from "adwo" 2014 as an example. In the coarse-grained traceability analysis, it is classified as "adwo" 2013 with a probability of 46.41% and also as "youmi" 2015 with a probability of 37.9%, suggesting a potential link. However, this contradicts the results in the family-level knowledge graph, which indicates that "adwo" 2014 originates from "adwo" 2013 and evolves to "kuguo" 2015, with no connection to "youmi" 2015 (Figure. 8).

To resolve these disputes, we delve into the specific behavior of "com.feixingqi.cxz.apk". By conducting SSG clustering, we extract the SSGs of the given sample and all malware in its candidate families. The SSGs are then grouped into clusters, with each cluster containing one SSG of the test sample. For instance, one cluster of "com.feixingqi.cxz.apk" and its candidates may involve connecting to the Internet, while another cluster attempts to access the user's phone number. This analysis provides evidence supporting the correctness of the coarse-grained malware traceability analysis, akin to the discovery of genes impacting debates in natural selection. Moreover, the abstracted SSGs offer additional information on the implementation of malicious behaviors, providing clear explanations for the evolution trends captured by security analysts.

Specifically, we extract the SSGs reflecting the traceability relation between "com.feixingqi.cxz.apk" ("adwo" 2014) and its candidates ("adwo" 2013 and "youmi" 2015) and then manually evaluate the accuracy of this explanation.

- 9 SSG clusters are generated while analyzing the traceability relation between"com.feixingqi.cxz.apk" (test sample) and the 21 malware in "youmi" 2015. Choosing two similar SSGs from the same cluster, one of them refers to the behavior of the test sample (Figure. 9 a)). Another one denotes the behavior of a malware named "com.hodanet.yanwenzi.apk" in "youmi" 2015 (Figure. 9 b)). As they all invoke the APIs "Calendar.getTimeZone" and "Calendar.getInstance" (the red nodes), malware will enable access to the calendar

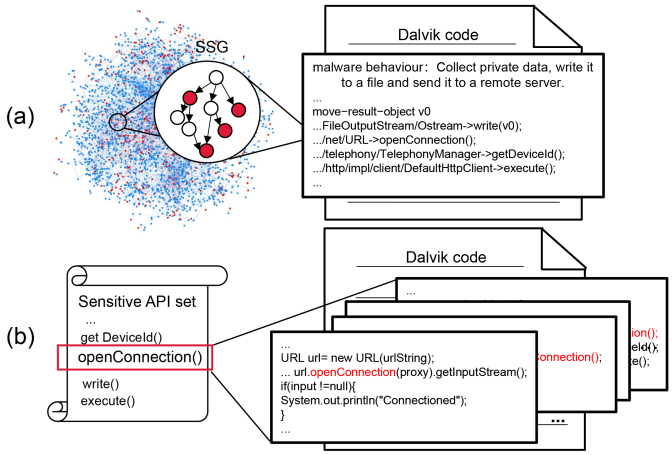


Fig. 10. Advantages of SSG in Traceability Analysis

and collect devices' timezone information. However, their complete operations are worked out via different APIs, such as "umeng.analytics" and "u.aly.bf.i".

- The SSGs from "com.feixingqi.cxz.apk" and 26 malware in "adwo" 2013 generate 6 clusters. One of those consists of an SSG from the test sample, two SSGs from "com.feixingqi.cxz.apk" and "com.azfrgame.sdcaiche.apk" in "adwo" 2013 respectively. These three SSGs mention the harmful behaviors apply "ConnectivityManager.getNetworkInfo", "NetworkInfo.getState" and "NetworkInfo.getSubtypeName" to steal current network information including network type, and connection state. We hence infer that the sample in "adwo" 2014 can be traced back to the behaviors taken by some samples in "adwo" 2013.

② **Traceability Relation of "jiagu":** In another case related to the "jiagu" family, the coarse-grained family-level traceability analysis in Figure. 5 indicates a close relation between "jiagu" 2021 and "dnotua" 2019, with a similarity of 71%. This suggests that malware from "jiagu" 2021 may have a probability of being categorized as "dnotua" 2019. To validate this, we select malware samples (e.g., "com.zbx.ct.tvzhibo.apk" as the test app) from "jiagu" 2021 and feed them to the pre-trained CNN classifier. However, the classifier identifies the test app as "youmi" 2017 and "youmi" 2014, rather than malware from "dnotua" 2019. This again verifies that malware traceability is more directly influenced by its behaviors.

We conduct an SSG-based investigation of the malicious behavior to resolve the conflicting results from the two different analysis levels. "com.zbx.ct.tvzhibo.apk" and its candidate families, "youmi" 2017 and "youmi" 2014, form 13 SSG clusters. Analyzing the SSGs in these clusters reveals a potential transition from "youmi" (both 2014 and 2017) to "jiagu" 2021. For example, when using identical APIs like "ClassLoader.loadClass" and "Class.getDeclaredMethod", they combine with other APIs in these SSGs to trigger malicious classes that install third-party applications. The behavior-level analysis enables us to obtain specific malicious behaviors of the samples, which are similar to those in their candidate families. This strengthens the accuracy of traceability results

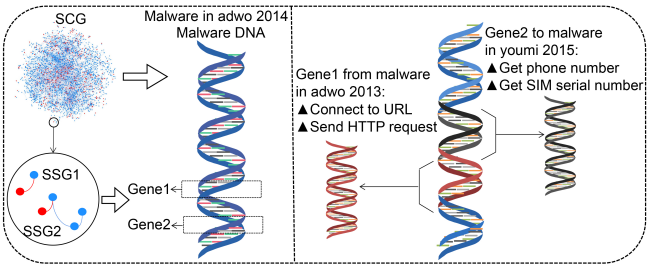


Fig. 11. Genome Constitution Analysis

and provides clear explanations for the evolution trends observed in practical analysis by security experts.

To demonstrate the effectiveness of using SSGs in traceability analysis, we illustrate a example in Figure. 10. In part (b), we observe that the behavior calls four sensitive APIs, including "write()", "openConnection()", "getDeviceId()", and "execute()", based on semantic features alone. However, without the structural information of the call relationship between APIs, accurately locating their usage in the source code and understanding their meaning becomes challenging. In contrast, in part (a), using SSGs allows for precise identification of the behavior's location in the source code and facilitates behavioral level analysis, confirming the traceability relationship between malware. In this example, we find that the behavior involves collecting users' private data, writing it into a file, and then sending it to a server, illustrating the advantages of using SSGs over shallow semantic features in traceability analysis.

2) Genome Constitution Analysis by Traceability Relation: This section focuses on explaining the root causes of malware evolution behind the traceable behaviors using a methodology inspired by genetic analysis in biological evolution.

① **Fundamentality:** We represent malware behaviors as genes (SSGs) and analyze their evolution. For example, on the left of Figure. 11, malware from adwo" 2014 has two genes, Gene1 and Gene2, which can be traced back to malware in "adwo" 2013 and "youmi" 2015 on the right of Figure. 11, respectively. These genes form the fundamental basis for exhibiting malicious behaviors.

② **Stability:** Malware genes Malware from "adwo" possesses a gene involving API "json.array" and "text.format.Time", which remains consistent from 2011 to 2014. This gene's persistent behavior is due to its relevance in monitoring time events [47], showing stability in the inheritance.

③ **Variability:** Malware evolves by modifying its SSGs through adding, deleting, or editing APIs, resulting in different behaviors and variants. This constant mutation is driven by attackers developing new malware and security experts implementing defense measures [48].

Runtime overhead estimation. We further measure the practical overhead of our work on a commodity workstation (8-core CPU, 32 GB RAM). As shown in Figure IV On our curated evolution dataset, constructing and pruning the SCG takes on average about 0.8 seconds per sample, SSG mining and feature extraction about 1.1 seconds, SSG embedding about 0.15 seconds, and evolution-link prediction about 0.05 seconds, yielding an end-to-end processing time of roughly

TABLE IV
AVERAGE RUNTIME PER SAMPLE FOR MAJOR COMPONENTS.

Component	Avg. time / sample
SCG construction & pruning	0.8 s
SSG mining & feature extraction	1.1 s
SSG embedding	0.15 s
Evolution-link prediction	0.05 s
End-to-end total	**2.1 s**

2.1 seconds per sample. The peak memory usage stays below 1 GB for all experiments. In practice, each app produces only tens of SSGs on average (at most a few hundred for very large apps), and these SSGs are encoded as compact vectors and compared via cosine similarity, rather than via expensive subgraph-isomorphism checks on raw graphs, which keeps the pipeline practical for large-scale offline deployment.

Because of the impact of attackers' and defenders' efforts, the malware in "jiagu" 2021 exhibits illegal operations similar to ransomware, while the malware from "youmi" 2017 utilizes Google Market Service (GMS) for installing third-party malware. However, changes in GMS availability and security experts' efforts in mitigation have led to the decline and disappearance of malware from the "youmi" family after 2017.

Overall, our genetic analysis methodology sheds light on understanding the evolutionary trends in malware behaviors, considering both the strategies employed by attackers and the defense mechanisms employed by security experts.

Remark: *This section demonstrates the soundness of our approach by automating traceability analysis and results explanation. The organic combination of family/sample-level and behavior-level analysis brings significant value, leveraging micro transitions between semantic and behavioral features. Sensitive subgraphs (SSGs), akin to biological genes, play a crucial role in pinpointing malicious behavior codes and remain well-preserved during analysis. The knowledge graph facilitates fine-grained analysis of the micro genome constitution in malware analysis, enhancing the precision and depth of the results.*

D. RQ3 - Practicability

To validate the practicality of our approach, we present a genetic analysis of cross-family malware traceability, showcasing representative phenomena identified from real-world malware data. The sample-level traceability relations and behavior-level interpretations were rigorously validated with analysis reports from other sources leveraging the explanation of the malware's sensitive subgraphs. The extensive validation process strengthens the practical applicability of our approach in effectively tracing malware relationships, making it a valuable tool for security professionals.

1) Genetic Analysis across Families: In § III-C1, we conduct a detailed examination of the evolution of "jiagu" and "adwo". "adwo" is a well-known and stable malware family, while "jiagu", the largest family in our dataset, constitutes nearly 30% of authenticated malware behaviors, linked to families like "youmi" and "dnotua". In this section, we conduct genetic analysis to investigate behavior changes in "jiagu" and "adwo", revealing the main reasons for traceability analysis changes. Due to space limitations, Figure. 12 displays "jiagu", "adwo", and related families, including "dnotua" (2017, 2019, 2021), "youmi" (2015, 2017), and "hiddenad" (2020) to demonstrate the practicality of our approach. On average, each family has 248 malware samples, with "jiagu" 2020 being the largest set with 1,198 samples, and "dnotua" 2021 containing 15 samples.

① **Relation with "dnotua"—"jiagu":** Apart from the legacy from 2017 to 2021 that implements unauthorized database changes (by loading APIs in classes like "SQLiteDatabase" and "CursorWindow"), family "dnotua" in the year 2019 delivers part of its genes to "jiagu" 2020. As is shown by the yellow lines in Figure. 12, the behavior manipulates malware to load classes into JVM (Java Virtual Machine) via API "ClassLoader.loadclass" and "Class.getDeclaredField", etc. Even if it seems to be a safe operation, the other APIs ("FileOutputStream.write") in the same gene could be activated to edit local files which may damage privacy.

② **Relation with "youmi"—"jiagu":** As early as 2015, "youmi", which was on the verge of extinction, had a tendency to switch to "jiagu" family in 2016 (purple line in Figure. 12). At that time, "jiagu" family was still in a precarious position, and it learned from "youmi" family's behavior of popping up advertisements without permission. The banished family "youmi" successfully insert its genes (the green lines in Figure. 12) into "jiagu" 2020, although it cannot be examined after the year 2017 by reasons of GMS and Dynamic SDK Manager. This notes that malware in "jiagu" 2020 gains the capability to open Asset files and extract figures by using "AssetManager.open" and "Resources.getAssets". At the same time, it has malware with harmful functional units that add malicious payload into the sandbox through "FileOutputStream.write" and its correlated APIs.

③ **Relation with "hiddenad"—"jiagu":** At the observation of "hiddenad" in the year 2020, we are concerned that its gene sticking into "jiagu" 2020 depicts a representation of achieving pictures from Asset files as same as "youmi". In Figure. 12, it stands for a mixture of genes from two malware families, which produces a hybrid malware (e.g., the little circle with two colors) in "jiagu". Additionally, as both of those two families are built in 2020, we manually check on a more specific time, such as the package time, of their malware for evolution reasoning. The empirical result proves a factor that 57% percent of malware in "hiddenad" 2020, which is created earlier than that in ''jiagu" 2020, determines the evolution mentioned above.

④ **Relation with "youmi"—"adwo":** Just like the disappeared "youmi" family is actually merged into "jiagu" and "adwo", which has evolved steadily for a long time, gradually disappeared in 2014. We guess it must have been transformed into other families or merged into a new family. Sure enough, its gene (the gray line on the picture) has been transferred to "youmi" in 2015. That is to say, in 2015, "youmi" learned from "adwo" the act of stealing the information of users' network

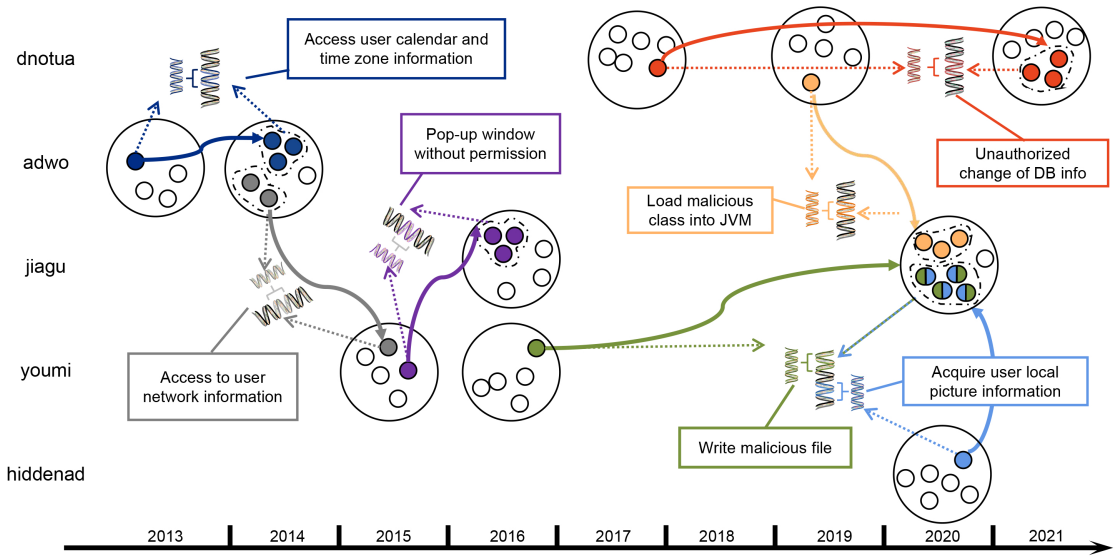


Fig. 12. Genetic Knowledge Graph of "jiagu" and "adwo"

hosts created by using APIs such as "java.net.InetAddress".

In brief, malware in "jiagu" 2020 has inherited various representative behaviors from the disappeared family "youmi", the developing family "dnotua", and the newly emerging family "hiddenad". "adwo" family is not as complicated as "jiagu" family. In 2014, "adwo" only evolved its main behavior to "youmi" family before disappearing. The security research of "jiagu" [49] also emphasizes the importance to deal with its malware with operations to hijack mobile devices or infect private documents. All of those demonstrations approve of the diversity and flexibility of malware family "jiagu", owing to the internal analysis of multiple granularity experiments and the external evaluation supported by security reports [50].

2) Post-analysis Validation with LLM: We involve a potential large language model (LLM)-based method for post-traceability analysis validation, and detail experimental guidelines to provide a feasible exploration direction for a fully automated system capable of perfecting engineering closed loops in the future.

① **Advanced Traceability Explanation:** This experiment involves obtaining provenance interpretation based on the target traceable relations, specifically clusters of sensitive subgraphs with high similarity. To achieve this, we backtrack the partial source code according to the obtained SSGs, extracting corresponding code fragments. These code fragments are then input into GPT using the few-shot prompt method to obtain their functional descriptions (detailed steps can be found on the website [43].).

② **Explanation Validation:** We compare the obtained functional descriptions with reports from other sources, such as VirusTotal, for cross-validating the captured traceability relations. This experiment aims to provide a relatively reliable cross-validation method by combining services based on different detection mechanisms. The goal is to identify potential false alarms, which may occur due to SSG being mistaken as a malicious behavior when it could actually be a result of code reuse in normal usage.

Prompt Template

You are an Android malware evolution expert.
Given two behavior summaries of sensitive subgraphs (SSGs), decide whether they are likely to be evolutionarily related.

Labels:

- "EVOLUTION": B is a variant or refinement of A (or vice versa), preserving the core malicious capability;
- "INDEPENDENT": clearly different malicious behaviors;
- "UNCERTAIN": not enough information.

Ruls:

1) Label "EVOLUTION" if the two SSGs share the same core malicious capability and one looks like a variant or refinement of the other (e.g., adds more data or encryption but keeps the main behavior).
2) Label "INDEPENDENT" if they implement clearly different malicious behaviors (e.g., one steals device IDs, the other sends premium SMS).
3) Label "UNCERTAIN" if the information is insufficient or ambiguous.
4) Base your decision on the main data accessed, communication channel, and malicious purpose.

Input : SSG A: <behavior summary of SSG A> | SSG B: <behavior summary of SSG B>
Output: JSON only:

{ "label": "EVOLUTION | INDEPENDENT | UNCERTAIN",
"reason": "..."}

Fig. 13. LLM-based validation prompt

③ **LLM-based validation prompt.** We design a simple, rule-guided prompt for LLM. The model is instructed to act as an Android malware evolution expert and, given two behavior summaries of sensitive subgraphs (SSGs), decide whether they are evolutionarily related. Concretely, as shown in Figure 13 the prompt asks the model to classify each pair into three discrete labels: EVOLUTION (the two SSGs share the same core malicious capability and one appears as a variant or refinement of the other, e.g., adding more data or encryption while preserving the main behavior), INDEPENDENT (they implement clearly different malicious behaviors, such as device ID theft vs. premium SMS fraud), or UNCERTAIN (the information is insufficient or ambiguous). The model is

required to base its decision on the main data accessed, the communication channel, and the overall malicious purpose, and to output a JSON object with a label field and a short textual reason explaining the decision.

Through the experiments conducted using the aforementioned method, we observed that the traceability relations of false detections only account for 16.7% compared with the correct detection. Moreover, we carefully analyzed the cases where validation failed (details on the website [43]), revealing that while the design of sensitive subgraphs exhibits excellent performance, it also leads to a loss of behavioral information. As a result, GPT is unable to fully verify specific malware behaviors in these cases.

Remark: *This section validates the real-world practicality of the approach. In the showcase, the knowledge graph reveals connections between 5 families through traceability analysis. Family/sample-level analysis identifies malware samples in "jiagu" and "adwo" linked to other families. Behavioral-level explanation confirms genes from "dnotua" and merged functions from "youmi" and "hiddenad", and "adwo" evolved its main behavior to "youmi" family before it disappeared.*

IV. Related Work

A. Malware Behaviors

In the face of malware threats, various works [3], [7], [10], [45], [47], [50]–[56] have focused on malware detection through feature analysis. Due to the limitations of independent and incomplete syntactic features like permissions and APIs, graph-based features have become prevalent in Android malware detection and behavior analysis [12], [37], [57]–[61]. Peta *et al.* [62] detect Android malware by constructing flowcharts based on operational code, while Ding *et al.* [63] create a feature vector for each function node in CFG that calls sensitive class packages to build features. DroidMiner [44] uses a two-level behavior graph, representing APKs through control flow and call graphs for malware classification. In this work, we express malware behaviors using sensitive subgraphs (SSGs) as part of our hierarchical analysis process. By inferring relationships based on similarity, we highlight the dominance of key APIs within the SSGs when deciphering evolutionary motivations.

B. Traceability Analysis

The conventional and direct approach for studying malware traceability is through the source code. Kalgutkar *et al.* [64] employ user-defined identifiers or strings for software traceability tasks, while Frantzeskou *et al.* [65] determine the source of a code piece by analyzing programmers' identifier writing habits. Zhao *et al.* [66] divide the code structure into subsets and analyze malware based on classes and methods within these subsets. Gonzalez *et al.* [67] propose author identification techniques to explore the authorship of Android binary code. However, the mentioned analyses rely on source code, and our objective extends beyond finding traceability based on the same author. Instead, we believe that as long as there are identical or similar semantic features or behavioral characteristics between malware samples, they exhibit traceability and evolutionary relationships.

C. Gene of Malware

The exploration of malware traceability necessitates the development of sensitive behavioral features, capitalizing on gene characteristics [21], [22]. Xiao *et al.* [23] utilize data streams to construct software genes for malicious software detection, while Han *et al.* [24] analyze malware using genetic binary sequences, including functions and information. Meng *et al.* [25] design detection with a distributed representation for each malware gene, refining gene sequences. While the initial motivation aligns with Genome [31], our focus lies in conducting multiple granularity knowledge graphs to uncover behavioral changes, moving away from labor-intensive manual analysis. By concentrating on gene properties, we aim to comprehend the gradual evolution of malware and reveal underlying incentives driving these changes.

V. Discussion

Threats to Vailidty. Our approach relies on a curated list of sensitive APIs. Although Android APIs evolve over time, we mitigate this threat by maintaining the list across versions and intersecting it with APIs observed in our 2011–2021 corpus; in practice, the evolution analysis is dominated by long-lived, frequently used sensitive APIs, and small changes in rare, version-specific calls have limited impact. In addition to API evolution, real world deployment also involves continuous malware ingestion and heavy obfuscation. Our pipeline can process newly arriving samples using the same static feature extraction and SSG construction, and the family year analysis can be updated periodically by inserting new samples and re evaluating evolution links within the affected scope. Heavy obfuscation and packing may hide parts of the payload and reduce the completeness of extracted SSGs, which can lead to missing evolution links. We partially mitigate this issue by anchoring on long lived sensitive API semantics and by aggregating multi sample evidence at the family year level so that incomplete signals from individual samples are less likely to dominate link decisions, but highly obfuscated cases remain challenging.

Reliance on static analysis and obfuscation. Our approach relies on static call graphs and API-level features, which limits visibility into code that only appears at runtime. Focusing on framework-level sensitive APIs makes SCG/SSG construction relatively robust to simple obfuscation (e.g., renaming, string encryption), but advanced techniques such as heavy reflection, dynamic loading, and packing can still hide parts of the malicious payload and cause us to miss some SSGs and evolution links. In practice, we partially mitigate the limitations of static analysis by (i) using framework-level sensitive APIs as anchors in SCG/SSG construction instead of easily obfuscated app-defined identifiers, and (ii) aggregating behaviors at the family–year level so that incomplete call graphs of individual samples are smoothed out by others.

TABLE V
POSITIONING OF OUR APPROACH AGAINST REPRESENTATIVE STUDIES ON ANDROID MALWARE TRACEABILITY/EVOLUTION.

Work (Year)	Target task	Granularity	Comparable
Tang et al. (2024) [68]	Variant evolution within families / family evolution (phylogenetic modeling)	Variant / family evolution	**Low**
Wu et al. (2025) [12]	Behavior understanding / description generation via behavior trajectory profiling	Behavior-level	**Low**
Gu et al. (2024) [69]	Malware detection (graph classification with representation learning)	App-level	**Low**
Wang et al. (2024) [70]	Family-aware malware analysis / classification via graph representation learning	App / family	**Low**
ArgusDroid (2023) [19]	Variant detection / variant identification using permission–API relations	family + Sample	**Middle**
FalDroid (2018) [37]	Familial classification and representative sample selection via frequent subgraph (fregraph) analysis	Family + Sample	**High**
DroidMiner (2014) [44]	Malware detection with automated mining and characterization of fine-grained malicious behaviors (threat modalities) via static analysis	Behavior-level + App-level	**Middle**
Gene-based (2017) [24]	Malware recognition / family-oriented characterization	Gene (statement-subsequence)	**Middle**
Ours	Traceability analysis via evolution-link prediction (multi-granularity: family–sample–behavior)	Family + Sample + Behavior	–

Baseline Suitability and Comparison Scope. We use Table V to position our work among representative studies on Android malware traceability and evolution and to clarify what is fairly comparable to our task. Most recent works are not suitable as direct baselines for our main setting, which is static traceability analysis via evolution link prediction. Phylogenetic modeling outputs phylogenetic trees or networks and follows evaluation criteria that differ from link prediction [68]. Behavior description methods focus on interpretability rather than inferring traceability links [12]. Recent graph neural network and representation learning methods mainly target malware detection or family classification, and hybrid static dynamic approaches require runtime traces that are not aligned with our static benchmark [69], [70]. Using these methods as baselines would require redefining the task or collecting additional dynamic datasets, which would lead to unfair comparisons. We therefore select ArgusDroid, FalDroid, and DroidMiner as baselines because they fit the static setting and provide the closest signals to our multi granularity traceability objective. ArgusDroid mines semantic relations between permissions and APIs [19]. FalDroid characterizes families and representative samples using frequent behavior subgraphs [37]. DroidMiner mines fine grained malicious behaviors through static analysis [44].

Limitations and Label Noise. Our evolution evaluation is conducted on a manually curated dataset with about 150 samples from 13 families, mainly because there is no publicly available dataset with explicit ground-truth labels for Android malware family evolution. We acknowledge potential scalability limitations and possible labeling bias since antivirus family labels can be noisy and inconsistent across engines. To mitigate this issue, we follow a conservative curation strategy based on multi-engine VirusTotal results and AVclass filtering, and two researchers manually merge and normalize family labels. For samples with large label disagreements, we further inspect package names, certificates, and existing analysis reports, and either assign a consistent family or discard the sample. After several rounds of manual filtering, we retain only high-confidence samples and correct abnormal timestamps using VirusTotal first-seen time.

Our traceability framework is also designed to be robust to residual family-label noise. It does not rely solely on AVClass-derived family labels. Instead, it is driven mainly by behavior-level semantics and temporal consistency. Sensitive Subgraphs (SSGs) act as "malware genes" that capture fine-grained malicious behaviors, and the evolution-link predictor uses SSG similarity together with the time axis to decide whether two family–year nodes are evolutionarily related. This design helps mitigate residual AVClass noise: even if some initial labels are imperfect, nodes whose SSG behaviors are inconsistent with the claimed family or time slice are unlikely to be connected by strong evolution links, thus reducing the impact of mislabeling on the overall knowledge graph structure.

Manual Analysis and Threshold Robustness. Certain steps in our approach require manual analysis, particularly in setting thresholds. We rely on analyzing results to establish accurate thresholds, which analyze necessary data for analysis. Identifying key malicious behaviors also involves manual analysis of aggregated SSGs and tracing variations, requiring background knowledge and massive efforts. However, we minimize expert efforts compared to other methods by transforming repeated program analysis costs into fine-grained descriptions of malware and its operations. Besides, our framework uses several similarity thresholds, such as cosine similarity and SSG similarity, which are selected on the curated dataset. We acknowledge that their optimal values may vary across datasets and future malware due to distribution shift. However, these thresholds are applied to similarity scores computed from static representations, and their role is to separate strong relations from weak relations when constructing evolution links, rather than to encode dataset-specific heuristics. In practice,

the thresholds can be calibrated with a lightweight procedure on a new dataset without changing the overall pipeline. For example, one can tune the thresholds on a small held-out calibration split by maximizing F1, or choose a percentile-based cutoff from the similarity score distribution within each family or time slice. Moreover, our evolution-link inference is constrained by temporal consistency and multi-granularity evidence, which helps reduce sensitivity to moderate threshold changes. We leave large-scale cross-dataset calibration as future work as more ground-truth evolution datasets become available.

Family Labeling and Traceability Analysis. Malware is labeled into families using AVclass [27], which may have inaccuracies as demonstrated in other studies [71]–[73]. Our approach primarily relies on SSG matching for traceability confirmation rather than family-level inspections, ensuring precise correction of erroneous evolutionary relations and maintaining internal validity.

External Validity Challenges. Although we have made efforts to balance the distribution of malware families to optimize the knowledge graph, quantitatively evaluating the real impact of our method is challenging due to the absence of benchmark data and comparable tools. Moreover, the lack of benchmark data poses another challenge as it makes it impossible to confirm all traceable relationships without ground truth of the provenance relationships. To address these limitations, our future research will concentrate on two main aspects: (1) Collaborating with corporate partners to conduct trials and gather more traceability relationships for validation; and (2) Enhancing the automated verification method for LLM-based traceability results to improve the accuracy and efficiency of the analysis process.

VI. Conclusion

We present a malware traceability analysis framework that merges semantic and behavioral malware features. Using static analysis and ML/DL techniques, we build a multi-grained knowledge graph, showcasing its applicability on large-scale data. Real-world analysis confirms the efficacy of tracking malware traceability, and we further explore the integration of static features to enhance the genetic methodology of traceability.

There are two interesting directions to explore in future work. First, we will enhance our framework with dynamic feature integration, incorporating runtime logs and behavioral traces to complement static analysis and improve traceability accuracy. Second, we plan to explore federated learning approaches for collaborative traceability, enabling knowledge sharing across different organizations while preserving data privacy. Third, inspired by recent advances in resource-aware edge AI training and large-model inference in wireless edge networks [74], [75], we plan to investigate how our traceability framework can be adapted to edge environments, including designing lightweight feature representations and offloading strategies that balance accuracy, latency, and resource constraints on real-world edge devices.

VII. Ethical Declaration

This research is conducted using publicly available datasets (e.g., AndroZoo, VirusTotal, AVclass, etc.), which contain only software binaries and metadata. No personally identifiable information (PII) or sensitive user data is involved.

References

[1] M. Sharif, V. Yegneswaran, H. Saidi, P. Porras, and W. Lee, "Eureka: A framework for enabling static malware analysis," in European Symposium on Research in Computer Security. Springer, 2008, pp. 481–500.

[2] C. Willems, T. Holz, and F. Freiling, "Toward automated dynamic malware analysis using cwsandbox," IEEE Security & Privacy, vol. 5, no. 2, pp. 32–39, 2007.

[3] D. Arp, M. Spreitzenbarth, M. Hubner, H. Gascon, K. Rieck, and C. Siemens, "Drebin: Effective and explainable detection of android malware in your pocket." in ISOC Network and Distributed System Security Symposium (NDSS), vol. 14, 2014, pp. 23–26.

[4] V. Rastogi, Y. Chen, and X. Jiang, "Droidchameleon: evaluating android anti-malware against transformation attacks," in Proceedings of the 8th ACM SIGSAC symposium on Information, computer and communications security (SIGSAC), 2013, pp. 329–334.

[5] B. Kolosnjaji, A. Zarras, G. Webster, and C. Eckert, "Deep learning for classification of malware system call sequences," in Australasian joint conference on artificial intelligence (AI). Springer, 2016, pp. 137–149.

[6] O. E. David and N. S. Netanyahu, "Deepsign: Deep learning for automatic malware signature generation and classification," in 2015 International Joint Conference on Neural Networks (IJCNN). IEEE, 2015, pp. 1–8.

[7] R. Feng, S. Chen, X. Xie, G. Meng, S.-W. Lin, and Y. Liu, "A performance-sensitive malware detection system using deep learning on mobile devices," IEEE Transactions on Information Forensics and Security, vol. 16, pp. 1563–1578, 2021.

[8] M. Ahmadi, D. Ulyanov, S. Semenov, M. Trofimov, and G. Giacinto, "Novel feature extraction, selection and fusion for effective malware family classification," in Proceedings of the sixth ACM conference on data and application security and privacy (CODASP), 2016, pp. 183–194.

[9] G. Meng, R. Feng, G. Bai, K. Chen, and Y. Liu, "Droidecho: an in-depth dissection of malicious behaviors in android applications," Cybersecurity, vol. 1, no. 1, pp. 1–17, 2018.

[10] Q. Qiao, R. Feng, S. Chen, F. Zhang, and X. Li, "Multi-label classification for android malware based on active learning," IEEE Transactions on Dependable and Secure Computing (TDSC), 2022.

[11] J. Chen, C. Wang, Z. Zhao, K. Chen, R. Du, and G.-J. Ahn, "Uncovering the face of android ransomware: Characterization and real-time detection," IEEE Transactions on Information Forensics and Security (TIFS), vol. 13, no. 5, pp. 1286–1300, 2017.

[12] C. Wu, S. Chen, J. Li, R. Chai, L. Fan, X. Xie, and R. Feng, "Beyond decision: Android malware description generation through profiling malicious behavior trajectory," ACM Trans. Softw. Eng. Methodol., vol. 34, no. 7, Aug. 2025. [Online]. Available: https://doi.org/10.1145/3715909

[13] Y. Huang, M. Qiao, F. Liu, X. Li, H. Gui, and C. Zhang, "Binary code traceability of multigranularity information fusion from the perspective of software genes," Computers & Security, vol. 114, p. 102607, 2022.

[14] R. Mohammadi, M. Hosseini, and R. Bahrami, "Uncovering security vulnerabilities through multiplatform malware analysis," Security and Privacy, vol. 8, no. 1, p. e455, 2025.

[15] Y. Li, H. Xiong, L. Kong, Z. Sun, H. Chen, S. Wang, and D. Yin, "Mpgraf: a modular and pre-trained graphformer for learning to rank at web-scale," in 2023 IEEE International Conference on Data Mining (ICDM). IEEE, 2023, pp. 339–348.

[16] Y. Li, H. Xiong, L. Kong, S. Wang, Z. Sun, H. Chen, G. Chen, and D. Yin, "Ltrgcn: Large-scale graph convolutional networks-based learning to rank for web search," in Joint European Conference on Machine Learning and Knowledge Discovery in Databases. Springer, 2023, pp. 635–651.

[17] Q. Lu, L. Zhu, X. Xu, J. Whittle, and Z. Xing, "Towards a roadmap on software engineering for responsible ai," arXiv preprint arXiv:2203.08594, 2022.

[18] J. Sun, Z. Xing, X. Peng, X. Xu, and L. Zhu, "Task-oriented api usage examples prompting powered by programming task knowledge graph," in 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, 2021, pp. 448–459.

[19] Y. Bai, S. Chen, Z. Xing, and X. Li, "Argusdroid: detecting android malware variants by mining permission-api knowledge graph," Science China Information Sciences, vol. 66, no. 9, p. 192101, 2023.

[20] R. Christian, S. Dutta, Y. Park, and N. Rastogi, "An ontology-driven knowledge graph for android malware," in Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 2435–2437.

[21] A. Pfeffer, C. Call, J. Chamberlain, L. Kellogg, J. Ouellette, T. Patten, G. Zacharias, A. Lakhotia, S. Golconda, J. Bay et al., "Malware analysis and attribution using genetic information," in 2012 7th International Conference on Malicious and Unwanted Software (MALCON). IEEE, 2012, pp. 39–45.

[22] B.-l. Zhao, Z. Shan, F.-d. Liu, B. Zhao, Y.-h. Chen, and W.-j. Sun, "Malware homology identification based on a gene perspective," Frontiers of Information Technology & Electronic Engineering (FITEE), vol. 20, no. 6, pp. 801–815, 2019.

[23] X. Da, L. Bohan, C. Baojiang, W. Xiaochen, and Z. Suoxing, "Malware prediction technique based on program gene," Chinese Journal of Network and Information Security, 2018.

[24] H. Jin, Z. Rongcai, S. Zhen, L. Fudong, Z. Bingling, M. Xi, and W. Hongyan, "Analyzing and recognizing android malware via semantic-based malware gene," in 2017 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC). IEEE, 2017, pp. 17–20.

[25] X. Meng, Z. Shan, F. Liu, B. Zhao, J. Han, H. Wang, and J. Wang, "Mcsmgs: malware classification model based on deep learning," in 2017 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery (CyberC). IEEE, 2017, pp. 272–275.

[26] VirusTotal, "Virustotal," 2004, https://www.virustotal.com/gui/home/upload.

[27] M. Sebastián, R. Rivera, P. Kotzias, and J. Caballero, "Avclass: A tool for massive malware labeling," in International symposium on research in attacks, intrusions, and defenses (RAID). Springer, 2016, pp. 230–253.

[28] K. Allix, T. F. Bissyandé, J. Klein, and Y. Le Traon, "Androzoo: Collecting millions of android apps for the research community," in 2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR). IEEE, 2016, pp. 468–471.

[29] M. Hayes, A. Walenstein, and A. Lakhotia, "Evaluation of malware phylogeny modelling systems using automated variant generation," Journal in Computer Virology, vol. 5, no. 4, pp. 335–343, 2009.

[30] T. Dumitras and I. Neamtiu, "Experimental challenges in cyber security: a story of provenance and lineage for malware," in 4th Workshop on Cyber Security Experimentation and Test (CSET), 2011.

[31] Y. Zhou and X. Jiang, "Dissecting android malware: Characterization and evolution," in 2012 IEEE symposium on security and privacy (SP). IEEE, 2012, pp. 95–109.

[32] S. Chen, M. Xue, L. Fan, S. Hao, L. Xu, H. Zhu, and B. Li, "Automated poisoning attacks and defenses in malware detection systems: An adversarial machine learning approach," computers & security, vol. 73, pp. 326–344, 2018.

[33] J. Jang, D. Brumley, and S. Venkataraman, "Bitshred: feature hashing malware for scalable triage and semantic analysis," in the 18th ACM conference on Computer and communications security (CCS), 2011, pp. 309–320.

[34] J. Jiang, S. Li, M. Yu, G. Li, C. Liu, K. Chen, H. Liu, and W. Huang, "Android malware family classification based on sensitive opcode sequence," in 2019 IEEE Symposium on Computers and Communications (ISCC). IEEE, 2019, pp. 1–7.

[35] L. Li, J. Gao, P. Kong, H. Wang, M. Huang, Y.-F. Li, and T. F. Bissyandé, "Knowledgezooclient: constructing knowledge graph for android," in 2020 35th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW). IEEE, 2020, pp. 73–78.

[36] H. C. Wu, R. W. P. Luk, K. F. Wong, and K. L. Kwok, "Interpreting tf-idf term weights as making relevance decisions," ACM Transactions on Information Systems (TOIS), vol. 26, no. 3, pp. 1–37, 2008.

[37] M. Fan, J. Liu, X. Luo, K. Chen, Z. Tian, Q. Zheng, and T. Liu, "Android malware familial classification and representative sample selection via frequent subgraph analysis," IEEE Transactions on Information Forensics and Security (TIFS), vol. 13, no. 8, pp. 1890–1905, 2018.

[38] M. E. Newman and M. Girvan, "Finding and evaluating community structure in networks," Physical review E, vol. 69, no. 2, p. 026113, 2004.

[39] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," Proceedings of the national academy of sciences (PNAS), vol. 105, no. 4, pp. 1118–1123, 2008.

[40] K. Tam, A. Feizollah, N. B. Anuar, R. Salleh, and L. Cavallaro, "The evolution of android malware and android analysis techniques," ACM Computing Surveys (CSUR), vol. 49, no. 4, pp. 1–41, 2017.

[41] R. You, Z. Zhang, Z. Wang, S. Dai, H. Mamitsuka, and S. Zhu, "Attentionxml: Label tree-based attention-aware deep model for high-performance extreme multi-label text classification," in Advances in Neural Information Processing Systems (NeurIPS), pp. 5812–5822.

[42] Z. Yan, H. Zhang, R. Piramuthu, V. Jagadeesh, D. DeCoste, W. Di, and Y. Yu, "Hd-cnn: hierarchical deep convolutional neural networks for large scale visual recognition," in Proceedings of the IEEE international conference on computer vision (ICCV), 2015, pp. 2740–2748.

[43] Y. Zhang, "Knowledge is power," 2025, https://github.com/ConfZ/KnowledgeIsPower.

[44] C. Yang, Z. Xu, G. Gu, V. Yegneswaran, and P. Porras, "Droidminer: Automated mining and characterization of fine-grained malicious behaviors in android applications," in European symposium on research in computer security (ESORICS). Springer, 2014, pp. 163–182.

[45] S. R. Moore, H. Ge, N. Li, and R. W. Proctor, "Cybersecurity for android applications: Permissions in android 5 and 6," International Journal of Human–Computer Interaction (IJHCI), vol. 35, no. 7, pp. 630–640, 2019.

[46] "Mobile & tablet android version market share worldwide," 2018, https://gs.statcounter.com/android-version-market-share/mobile-tablet/worldwide/2018.

[47] F. Wei, Y. Li, S. Roy, X. Ou, and W. Zhou, "Deep ground truth analysis of current android malware," in International conference on detection of intrusions and malware, and vulnerability assessment (DIMVA). Springer, 2017, pp. 252–276.

[48] Symantec, "Androguard," 2016, http://www.symantec.com/connect/blogs/future-mobile-malware.

[49] G. Suarez-Tangil and G. Stringhini, "Eight years of rider measurement in the android malware ecosystem: evolution and lessons learned," arXiv preprint arXiv:1801.08115, 2018.

[50] D. S. Keyes, B. Li, G. Kaur, A. H. Lashkari, F. Gagnon, and F. Massicotte, "Entroplyzer: Android malware classification and characterization using entropy analysis of dynamic characteristics," in 2021 Reconciling Data Analytics, Automation, Privacy, and Security: A Big Data Challenge (RDAAPS). IEEE, 2021, pp. 1–12.

[51] M. Wadkar, F. Di Troia, and M. Stamp, "Detecting malware evolution using support vector machines," Expert Systems with Applications (ESWA), vol. 143, p. 113022, 2020.

[52] K. Rieck, P. Trinius, C. Willems, and T. Holz, "Automatic analysis of malware behavior using machine learning," Journal of computer security, vol. 19, no. 4, pp. 639–668, 2011.

[53] R. Feng, S. Chen, X. Xie, L. Ma, G. Meng, Y. Liu, and S.-W. Lin, "Mobidroid: A performance-sensitive malware detection system on mobile platform," in 2019 24th International Conference on Engineering of Complex Computer Systems (ICECCS), 2019, pp. 61–70.

[54] R. Feng, Y. Liu, and S. Lin, "A performance-sensitive malware detection system on mobile platform," in Formal Methods and Software Engineering, Y. Ait-Ameur and S. Qin, Eds. Cham: Springer International Publishing, 2019, pp. 493–497.

[55] R. Feng, J. Q. Lim, S. Chen, S.-W. Lin, and Y. Liu, "Seqmobile: An efficient sequence-based malware detection system using rnn on mobile devices," in 2020 25th International Conference on Engineering of Complex Computer Systems (ICECCS), 2020, pp. 63–72.

[56] Y. Li, R. Feng, S. Chen, Q. Guo, L. Fan, and X. Li, "Iconchecker: Anomaly detection of icon-behaviors for android apps," in 2021 28th Asia-Pacific Software Engineering Conference (APSEC), 2021, pp. 202–212.

[57] X. Zhiwu, K. Ren, and F. Song, "Android malware family classification and characterization using cfg and dfg," in 2019 International Symposium on Theoretical Aspects of Software Engineering (TASE). IEEE, 2019, pp. 49–56.

[58] Y. Qu, X. Guan, Q. Zheng, T. Liu, L. Wang, Y. Hou, and Z. Yang, "Exploring community structure of software call graph and its applications in class cohesion measurement," Journal of Systems and Software (JSS), vol. 108, pp. 193–210, 2015.

[59] M. Hassen and P. K. Chan, "Scalable function call graph-based malware classification," in Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy (CODASPY), 2017, pp. 239–248.

[60] S. Li, M. Ge, R. Feng, X. Li, and K. Y. Lam, "Automatic detection and analysis towards malicious behavior in iot malware," in 2023 IEEE International Conference on Data Mining Workshops (ICDMW), 2023, pp. 1332–1341.

[61] R. Feng, S. Li, S. Chen, M. Ge, X. Li, and X. Li, "Unmasking the lurking: Malicious behavior detection for iot malware with multi-label classification," ser. LCTES 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 95–106. [Online]. Available: https://doi.org/10.1145/3652032.3657577

[62] A. Pektaş and T. Acarman, "Learning to detect android malware via opcode sequences," Neurocomputing, vol. 396, pp. 599–608, 2020.

[63] Y. Ding, S. Zhu, and X. Xia, "Android malware detection method based on function call graphs," in International Conference on Neural Information Processing (ICONIP). Springer, 2016, pp. 70–77.

[64] V. Kalgutkar, N. Stakhanova, P. Cook, and A. Matyukhina, "Android authorship attribution through string analysis," in Proceedings of the 13th International Conference on Availability, Reliability and Security (ARES), 2018, pp. 1–10.

[65] G. Frantzeskou, S. G. MacDonell, E. Stamatatos, S. Georgiou, and S. Gritzalis, "The significance of user-defined identifiers in java source code authorship identification," arXiv preprint arXiv:2101.12384, 2021.

[66] L. Zhao, J. Wang, Y. Chen, F. Wu, Y. Liu et al., "Famdroid: learning-based android malware family classification using static analysis," arXiv preprint arXiv:2101.03965, 2021.

[67] H. Gonzalez, N. Stakhanova, and A. A. Ghorbani, "Authorship attribution of android apps," in Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy (CODASPY), 2018, pp. 277–286.

[68] L. Tang, X. Chen, S. Wen, L. Li, M. Grobler, and Y. Xiang, "Demystifying the evolution of android malware variants," IEEE Transactions on Dependable and Secure Computing, vol. 21, no. 4, pp. 3324–3341, 2023.

[69] J. Gu, H. Zhu, Z. Han, X. Li, and J. Zhao, "Gsedroid: Gnn-based android malware detection framework using lightweight semantic embedding," Computers & Security, vol. 140, p. 103807, 2024.

[70] Z. Wang, K. Zeng, J. Wang, and D. Li, "Fagnet: Family-aware-based android malware analysis using graph neural network," Knowledge-Based Systems, vol. 289, p. 111531, 2024.

[71] P. Kotzias, J. Caballero, and L. Bilge, "How did that get in my phone? unwanted app distribution on android devices," in 2021 IEEE Symposium on Security and Privacy (SP). IEEE, 2021, pp. 53–69.

[72] A. Küchler, A. Mantovani, Y. Han, L. Bilge, and D. Balzarotti, "Does every second count? time-based evolution of malware behavior in sandboxes." in 28th Annual Network and Distributed System Security Symposium (NDSS), 2021.

[73] M. Scalas, D. Maiorca, F. Mercaldo, C. A. Visaggio, F. Martinelli, and G. Giacinto, "On the effectiveness of system api-related information for android ransomware detection," Computers & Security, vol. 86, pp. 168–182, 2019.

[74] Z. Lyu, Y. Li, G. Zhu, J. Xu, H. V. Poor, and S. Cui, "Rethinking resource management in edge learning: A joint pre-training and fine-tuning design paradigm," IEEE Transactions on Wireless Communications, 2024.

[75] Z. Lyu, M. Xiao, J. Xu, M. Skoglund, and M. Di Renzo, "The larger the merrier? efficient large ai model inference in wireless edge networks. arxiv 2025," arXiv preprint arXiv:2505.09214, 2025.